

Java para Web

Introdução, Servlets, JSP e Apache Tomcat

Mauricio Duailibi Neto

9 de junho de 2026

O fim do Terminal Preto

Até hoje, nossos sistemas funcionavam apenas na tela preta do terminal do VS Code. O usuário precisava ter o Java instalado e rodar o programa localmente.

A partir de agora, nosso código Java vai rodar nos bastidores de um **Servidor**. O usuário final vai acessar o nosso sistema através de um Navegador Comum (Chrome, Edge, Safari), de qualquer lugar do mundo.

Vamos transformar nossas lógicas e objetos em páginas Web dinâmicas.

A Arquitetura Cliente-Servidor

A internet é baseada em um diálogo simples:

- 1 **Request (Requisição):** O Cliente (Navegador) pede algo. Ex: "Me dê a página de calcular IMC".
- 2 **Processamento:** O Servidor (onde o Java mora) recebe o pedido, faz os cálculos, acessa banco de dados, e monta uma resposta.
- 3 **Response (Resposta):** O Servidor devolve a página montada para o navegador exibir ao usuário.

Por que usar Java na Web?

Web Estática (HTML/CSS puro): São como panfletos digitais. O arquivo HTML está pronto no servidor e é entregue exatamente igual para todos os usuários. Não há lógica, não há cálculo.

Web Dinâmica (Java Web): A página é construída em tempo real de acordo com quem está acessando. Se você logar no banco, verá seu saldo. Se eu logar, verei o meu. O Java gera um HTML personalizado para cada requisição.

Os conceitos fundamentais

Para criar aplicações Web em Java, usaremos um conjunto de ferramentas e conceitos padrão da indústria:

- Pacotes (Packages)
- Maven
- Apache Tomcat
- Servlets
- JSP (JavaServer Pages)

Vamos entender o papel de cada um deles antes de colocarmos a mão na massa.

O que é um Pacote (Package)?

As pastas do sistema

Até agora, colocávamos todos os arquivos `.java` juntos. Em um projeto real com centenas de classes, isso vira um caos.

Um **Package** é simplesmente uma pasta virtual que organiza nossas classes por contexto. Em Java, usamos o domínio da empresa invertido como padrão de nomenclatura.

Exemplo: `br.com.escola.model`

- A classe `Aluno` fica dentro dessa estrutura.
- Evita conflito de nomes (Podemos ter uma classe `Login` do cliente e outra `Login` do administrador em pacotes diferentes).

O que é o Maven?

O Gerenciador da Obra

Construir um projeto Web exige muitas configurações e bibliotecas de terceiros. Fazer download manual de arquivos `.jar` e configurá-los é coisa do passado.

O **Maven** é um gerenciador de projetos. Ele possui um arquivo central chamado `pom.xml`.

- Se precisarmos conectar ao banco de dados, escrevemos o nome do conector no `pom.xml`.
- O Maven vai até a internet, baixa a versão correta, e coloca no nosso projeto automaticamente.
- Ele também padroniza a estrutura de pastas do nosso projeto.

O que é o Apache Tomcat?

O Servidor Web (A Casa da Aplicação)

O nosso código Java puro não sabe conversar com a internet. Ele não entende requisições HTTP do Chrome.

O **Tomcat** é um "Web Container". Ele é um programa que roda na nossa máquina (e nos servidores reais) escutando a porta da internet (geralmente a porta 8080).

- Ele recebe o pedido do usuário.
- Ele traduz esse pedido para o Java.
- Ele entrega o resultado de volta para o usuário.

O que é um Servlet?

O Controlador Lógico

Um **Servlet** é uma Classe Java especial. Ele é a única classe capaz de receber a requisição que veio do Tomcat, processar regras de negócio e tomar decisões.

Se o usuário preencher um formulário de cálculo de IMC e clicar em "Enviar", quem recebe esses dados capturados (Peso e Altura) é o nosso Servlet. É nele que escreveremos o código Java pesado.

O que é um JSP?

A Tela do Usuário

JSP significa *JavaServer Pages*. Ele é essencialmente um arquivo HTML comum, mas com um superpoder: ele aceita variáveis do Java misturadas no meio do texto.

Enquanto o **Servlet** faz o cálculo nos bastidores, o **JSP** é a resposta visual. O Servlet calcula o IMC e envia o número para o JSP. O JSP pega o número, coloca dentro de uma tag HTML bonita e exibe para o usuário.

O caminho do dado

- 1 O usuário acessa `index.jsp` e vê um formulário.
- 2 Ele digita Peso e Altura e clica em Calcular.
- 3 O formulário dispara uma requisição para o `ImcServlet.java`.
- 4 O Servlet captura os dados, converte para números, e calcula o IMC usando Orientação a Objetos.
- 5 O Servlet anexa o resultado na requisição e redireciona o usuário para `resultado.jsp`.
- 6 O `resultado.jsp` exibe a página final formatada.

Preparando o Ambiente de Trabalho

Requisitos

Vocês já possuem a base instalada nas máquinas:

- VS Code
- OpenJDK 17
- Extension Pack for Java

Agora precisamos configurar a ponte para a Web: Instalar o Servidor Tomcat e a extensão para gerenciá-lo.

Instalando o Apache Tomcat

Passo a Passo

- 1 Acesse: `tomcat.apache.org/download-10.cgi`
- 2 Na seção "Core", baixe o arquivo **zip** (não o instalador de Windows).
- 3 Extraia a pasta `apache-tomcat-10.x.x` no seu disco C: (ex: `C:/Tomcat10`).
- 4 *Atenção: Apenas extraia, não precisa executar nada dentro da pasta ainda.*

A extensão do Servidor

- 1 Abra o VS Code.
- 2 Vá na aba de Extensões.
- 3 Pesquise por **Community Server Connectors** (da Red Hat).
- 4 Clique em Install.
- 5 Após instalado, você verá um novo ícone na barra esquerda ou uma aba chamada "SERVERS" no Explorer.

Configuração final

- 1 Na aba SERVERS, clique no botão + (Create New Server).
- 2 Se perguntar se quer fazer download, diga **No, use local server**.
- 3 Navegue pelas pastas e selecione a pasta do Tomcat que você extraiu no C:/Tomcat10.
- 4 Pronto! O Tomcat está acoplado ao seu VS Code e pronto para rodar nossos projetos.

Criando o Projeto Maven Web

Gerando a estrutura

- 1 Pressione Ctrl + Shift + P (Command Palette).
- 2 Digite Maven: Create Maven Project.
- 3 Selecione o archetype: maven-archetype-webapp.
- 4 Escolha a versão (pode ser a última).
- 5 Digite o Group Id (nosso pacote base): `br.com.aula`.
- 6 Digite o Artifact Id (nome do projeto): `calculadora-imc`.
- 7 Selecione a pasta onde salvar e dê Enter.

Onde colocar os arquivos

O Maven cria várias pastas. É fundamental saber onde trabalhar:

- `src/main/java/`: Aqui criaremos nossos pacotes e todas as nossas classes `.java` (nossos Servlets). *(Se essa pasta não existir, pode criá-la manualmente).*
- `src/main/webapp/`: Aqui é a pasta pública. Onde colocamos os arquivos `.jsp`, `.html`, imagens e CSS.
- `pom.xml`: O arquivo raiz de configurações do Maven.

Configurando o pom.xml

Adicionando a API de Servlets

Abra o arquivo pom.xml. Procure a tag <dependencies> e adicione a dependência do Jakarta Servlet. Precisamos dela para que o Java entenda as requisições da Web.

```
<dependencies>

  <dependency>
    <groupId>jakarta.servlet</groupId>
    <artifactId>jakarta.servlet-api</artifactId>
    <version>5.0.0</version>
    <scope>provided</scope>
  </dependency>

</dependencies>
```

O scope "provided" diz que quem vai fornecer essa biblioteca na hora de rodar é o próprio Tomcat.

Mão na massa!

Vamos construir nossa aplicação passo a passo. A sequência de desenvolvimento Web clássica é:

- 1 Criar a Tela de Entrada (JSP).
- 2 Criar o Controlador (Servlet).
- 3 Criar a Tela de Resultado (JSP).

Passo 1: O Formulário de Entrada

Arquivo: src/main/webapp/index.jsp

Apague o conteúdo padrão e crie um formulário simples. A tag action diz para onde os dados serão enviados.

```
<html>
<body>
  <h2>Calculadora de IMC</h2>
  <form action="calcularImc" method="POST">
    <label>Peso (kg):</label>
    <input type="text" name="pesoCampo">
    <br><br>

    <label>Altura (m):</label>
    <input type="text" name="alturaCampo">
    <br><br>

    <button type="submit">Calcular</button>
  </form>
</body>
</html>
```

Passo 2: Criando o Pacote e o Servlet

Configurando o ambiente Java

- 1 Clique com o botão direito na pasta `src/main/java`.
- 2 Crie uma nova pasta chamada `br/com/aula/controller` (este é o nosso Pacote).
- 3 Dentro desta pasta, crie um arquivo chamado `ImcServlet.java`.

Arquivo: ImcServlet.java

Para ser um Servlet, a classe deve dar `extends HttpServlet` e usar a anotação `@WebServlet` para definir a sua rota URL (a mesma do formulário).

```
package br.com.aula.controller;

import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

@WebServlet("/calcularImc")
public class ImcServlet extends HttpServlet {

}
```

Capturando a Requisição

Sobrescrevendo o método doPost

Dentro do Servlet, criamos o método doPost (pois o formulário usou o método POST). Ele recebe o pedido (request) e prepara a resposta (response).

```
@Override
protected void doPost(HttpServletRequest request, HttpServletResponse
    response) {

    try {

        String pesoStr = request.getParameter("pesoCampo");
        String alturaStr = request.getParameter("alturaCampo");

    } catch (Exception e) {

    }

}
```

Cálculo e Conversão

Tudo que vem da web chega como String. Precisamos converter para double antes de calcular a matemática real.

```
double peso = Double.parseDouble(pesoStr);
double altura = Double.parseDouble(alturaStr);

double imc = peso / (altura * altura);

String classificacao;
if (imc < 18.5) classificacao = "Abaixo do peso";
else if (imc < 25) classificacao = "Peso normal";
else if (imc < 30) classificacao = "Sobrepeso";
else classificacao = "Obesidade";
```


Enviando os dados para a próxima tela

Despachando a requisição

O Servlet calculou, agora ele pendura os resultados na requisição e avisa o servidor para repassar a bola para o arquivo `resultado.jsp`.

```
request.setAttribute("resultadoImc", imc);
request.setAttribute("resultadoClasse", classificacao);

request.getRequestDispatcher("resultado.jsp").forward(request,
    response);

} catch (Exception e) {
    System.out.println("Erro no calculo.");
}
```

O Servlet Completo

```
package br.com.aula.controller;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.*;

@WebServlet("/calcularImc")
public class ImcServlet extends HttpServlet {
    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse
        response) {
        try {
            String pesoStr = request.getParameter("pesoCampo");
            String alturaStr = request.getParameter("alturaCampo");
            double peso = Double.parseDouble(pesoStr);
            double altura = Double.parseDouble(alturaStr);

            double imc = peso / (altura * altura);
            String classificacao;
            if (imc < 18.5) classificacao = "Abaixo do peso";
            else if (imc < 25) classificacao = "Peso normal";
            else classificacao = "Acima do peso";

            request.setAttribute("resultadoImc", imc);
            request.setAttribute("resultadoClasse", classificacao);
            request.getRequestDispatcher("resultado.jsp").forward(request,
                response);
        } catch (Exception e) {}
    }
}
```

Passo 3: A Tela de Resultado

Arquivo: src/main/webapp/resultado.jsp

Agora criamos o arquivo JSP final. Usamos a notação Expression Language (EL) que são o cifrão e as chaves `${}` para resgatar os atributos pendurados pelo Servlet.

```
<html>
<body>
    <h2>Resultado do seu IMC</h2>

    <p>Seu IMC exato    : ${resultadoImc}</p>

    <h3>Classifica    o : ${resultadoClasse}</h3>

    <br>
    <a href="index.jsp">Voltar e calcular novamente</a>
</body>
</html>
```

O Teste Final

- 1 Clique com o botão direito no seu projeto ou no `pom.xml`.
- 2 Se houver a opção "Run on Server", clique nela.
- 3 Ou vá na aba SERVERS (do Tomcat), clique com o botão direito no Servidor e escolha "Start".
- 4 Após iniciar, clique com o botão direito no Servidor novamente, vá em "Add Deployment" e selecione a pasta target do seu projeto compilado.
- 5 Abra o navegador e digite: `http://localhost:8080/calculadora-imc`

Sistema de Aprovação Escolar

Sua missão para essa semana é replicar essa estrutura em um projeto novo chamado `sistema-notas`.

Requisitos:

- 1 O `index.jsp` deve ter um formulário pedindo o Nome do Aluno, Nota 1 e Nota 2.
- 2 O Servlet `NotasServlet` deve ser mapeado na URL `/avaliar`.
- 3 O Servlet deve pegar os dados, converter as notas, calcular a média simples.
- 4 Regra: Se a média for ≥ 7.0 , o aluno está Aprovado. Senão, Reprovado.
- 5 Envie o nome, a média e o status para o `resultado.jsp`.
- 6 O `resultado.jsp` deve imprimir: "O aluno [Nome] teve média [Media] e está [Status]".

O Nascimento do Desenvolvedor Backend

Hoje foi uma aula densa de infraestrutura. Vocês aprenderam que a internet não passa de arquivos de texto conversando entre si.

Entender o fluxo da requisição (Formulário -> Servlet -> Processamento -> JSP) é a habilidade mais importante do Java Web. Pratiquem o desafio, enfrentem os erros do Tomcat e construam a base sólida para criarmos sistemas gigantescos!

Boa semana de código a todos!